VECTOR

Fbl Updater TC3xx

Technical Reference

Hardware specific information

Version 0.1

Authors	Sebastian Loos
Status	Not Released

Document Information

History

Author	Date	Version	Remarks
Sebastian Loos	2018-010-18	1.00	Initial Version

Reference Documents

No.	Source	Title	Version
[1]	Vector	TechnicalReference_FBL_Updater.pdf	



Caution
We have configured the programs in accordance with your specifications in the questionnaire. Whereas the programs do support other configurations than the one specified in your questionnaire, Vector´s release of the programs delivered to your company is expressly restricted to the configuration you have specified in the questionnaire.

© 2018 Vector Informatik GmbH

Contents

1 Introduction 5

2 Reset-Safe Update Process 6
2.1 Theory of operation 6
2.1.1 Addresses 6
2.1.2 ABM and BMHDi_x 6
2.1.3 Protection 7
2.2 Write Sequence for UCB BMHDs 7
2.3 Variants 8
2.3.1 Variant A: Replace the first BMHD for Updater 8
2.3.2 Variant B: Use second BMHD for Updater 9
2.3.3 Variant C: Use ABM for FBL 10

3 Integration 12
3.1 Addressing 12
3.2 Initializations 12
3.3 Hooks 12
3.4 BMHD content 13
3.4.1 BMHD Generator Script 13
3.5 Flash Driver settings 14
3.6 Configuration 14
3.7 Demonstration Code 14

4 Abbreviations 16

5 Contact 17

Illustrations

Figure 2-1 Figure 2-2 ABM and BMHDi_x 7
Figure 2-3 Variant A: Replace the first BMHD for Updater 9
Figure 2-4 Variant B: Use second BMHD for Updater 10
Figure 2-5 Variant C use ABMHD for Bootloader 11
Figure 3-1 Typical BMHD content 13
Figure 3-2 Screenshot from batch file 14
Figure 3-3 Disable Protection flow 15

Tables

Table 2-1 Typical BMHD locations 6
Table 3-1 Possible (expected) situations on start of the Updater 15

1 Introduction

This document will give a brief overview on the Tricore Aurix Plus (TC3xx) family specific aspects of the FBL Updater, A general description of the Updater can be found in [1].

The handling of the BMHD is different for Aurix (TC2xx) and Aurix Plus (TC3xx). Therefore there is a dedicated component `FblUpd_Ap_TC3xx` which supports Aurix Plus derivatives. It is not compatible to the TC2xx (Aurix) solution in `FblUpd_Ap_TricoreAurix`.

© 2018 Vector Informatik GmbH

2 Reset-Safe Update Process

Reset-Safety describes that the Updater will in each case finish the Update process – even if it is interrupted e.g. by a reset. This requires means to start up the Updater after a reset without having a valid Bootloader present.

2.1 Theory of operation

The Tricore uses the Boot Mode Headers (BMHD) to decide the starting address after a reset. Before an update the BMHD will point to the Bootloaders starting address. The Updater will now

- change the BMHD to point on his own starting address,
- erase and replace the Bootloader
- change BMHD to point to the Bootloader again.

If the update process is interrupted by e.g. a power-down reset, it will be repeated after the reset until it is finished successfully.



Caution
Make sure that always at least one valid BMHD exists in your device. Otherwise it may get locked and become inaccessible for the debugger.

Because of the special design of the BMHDs, an incomplete write to a BMHD will not cause a fatal situation. In each case there is a backup (_COPY) of the altered entry (_ORIG).

2.1.1 Addresses

The BMHDs are stored in a separate memory, the UCB (User configuration block) area. Table 2-1 shows the typical locations.

Address	Content	Order of Evaluation
0xAF40 0000 - 0xAF40 01FF	UCB00_BMHD0_ORIG	0
0xAF40 0200 - 0xAF40 03FF	UCB01_BMHD1_ORIG	2
0xAF40 0400 - 0xAF40 05FF	UCB02_BMHD2_ORIG	4
0xAF40 0600 - 0xAF40 07FF	UCB03_BMHD3_ORIG	6
0xAF40 0000 - 0xAF40 11FF	UCB08_BMHD0_COPY	1
0xAF40 0200 - 0xAF40 13FF	UCB09_BMHD1_COPY	3
0xAF40 0400 - 0xAF40 15FF	UCB10_BMHD2_COPY	5
0xAF40 0600 - 0xAF40 17FF	UCB11_BMHD3_COPY	7

Table 2-1 Typical BMHD locations

2.1.2 ABM and BMHD*i*_*x*

On startup, the BMHDs are evaluated by the firmware. The firmware checks sequentially BMHD0 … BMHD3 for validity. The first valid BMHD will be used to determine the startup address (see Figure 2-2). A BMHD can reference a “ABMHD” structure in the program flash.

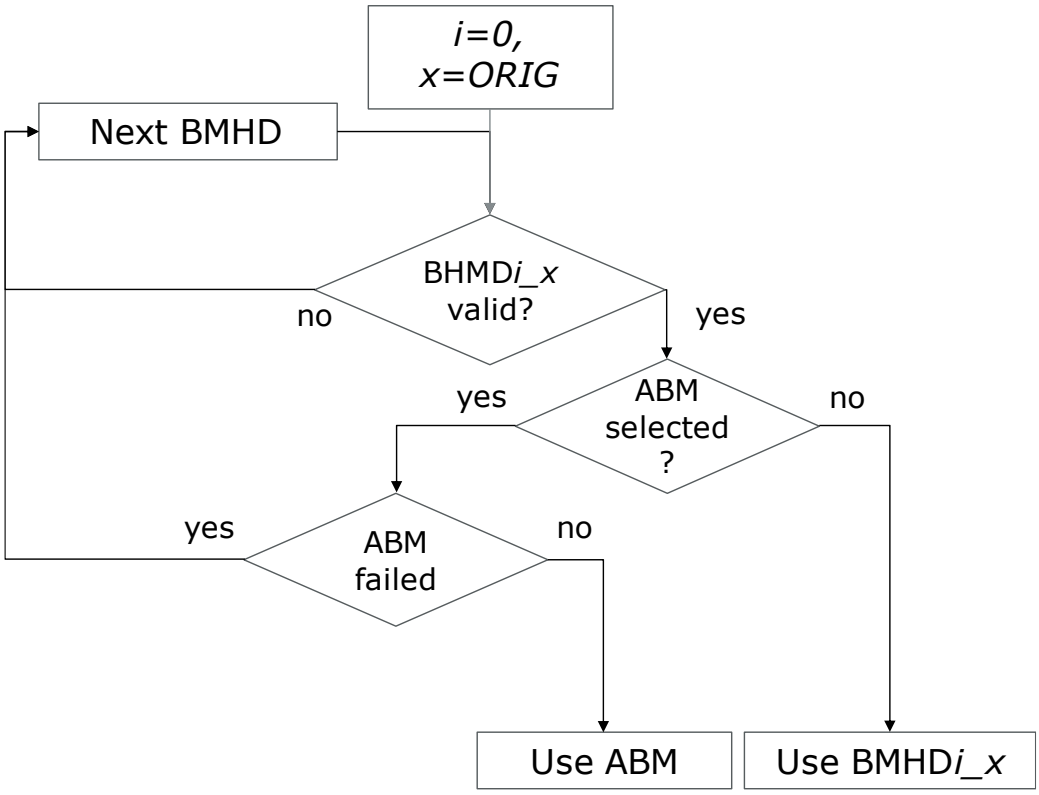


Figure 2-1 Figure 2-2 ABM and BMHD*i_x*

As shown in Table 2-1, BMHD0_ORIG will be evaluated first, followed by BMHD0_COPY, followed by BMHD1_ORIG etc.

2.1.3 Protection

BMHD0 (_ORIG and _COPY) can be protected by a 256-bit password. If a password is present, it is not possible to

- Read the password from the flash
- Erase the BMHD.

To remove the password protection, the “Disable Protection” sequence is required.

2.2 Write Sequence for UCB BMHDs

According to Infineon, the correct sequence for writing a BMHD is

a) Check HF_CONFIRM0 for BMHD*i*_ORIG in state CONFIRMED or UNLOCKED

b) Check HF_CONFIRM0 for BMHD*i*_COPY in state CONFIRMED or UNLOCKED

c) If CONFIRMED do Disable Protection sequence

d) Erase BMHD*i*_COPY

Write BMHD*i*_COPY: Write whole UCB: Write PWs, write UNLOCKED. Write reserved or unused bytes to zero.

e) Verify written data (read back)

f) Write `CONFIRMED`

g) Verify written `CONFIRMED`

h) Repeat from step d) For `BMHDi_ORIG`

The Flash Driver usually verifies all written data, therefore steps d) and g) are not implemented as a separate step.



Caution
If writing a BMHD value, make sure that the **complete** UCB Block is written with **valid** data. If e.g. the passwords PW*i* are not fully written, a ECC failure may occur during startup which stops the startup sequence. Even if the STAD and Checksum fields are written valid. This will make the device inaccessible by the debugger.

It is possible to skip step f) and g). It is not required to set the BMHD to `CONFIRMED`, setting to `UNLOCKED` is sufficient. BMHDs in state `UNLOCKED` can be erased and written without a Disable Protection sequence.



Caution
Writing passwords different than zeros to UCB00_BMHD0 enables protection for the BMHD. **With enabled protection, reading back the passwords is not possible**. Further it is not possible to erase/write the BMHD without executing a Disable Protection Sequence. For this sequence the programmed password needs to be known.

2.3 Variants

There are several variants how the BMHDs can be used to achieve a reset-safe update process. Three of them are explained in the following. Of course, other solutions are also possible (e. g. a mix of A and C).

The demonstration code contains a solution for Variant A.

2.3.1 Variant A: Replace the first BMHD for Updater

This is the recommended solution. It is the most flexible variant because it makes no assumption on the memory mapping. As shown in Figure 2-3, the BMHD is changed to the Updaters starting address prior to erasing and writing the new Bootloader.

The drawback of this solution is, that writing the UCB area is required which can cause a bricked device if the configuration is not completely right.

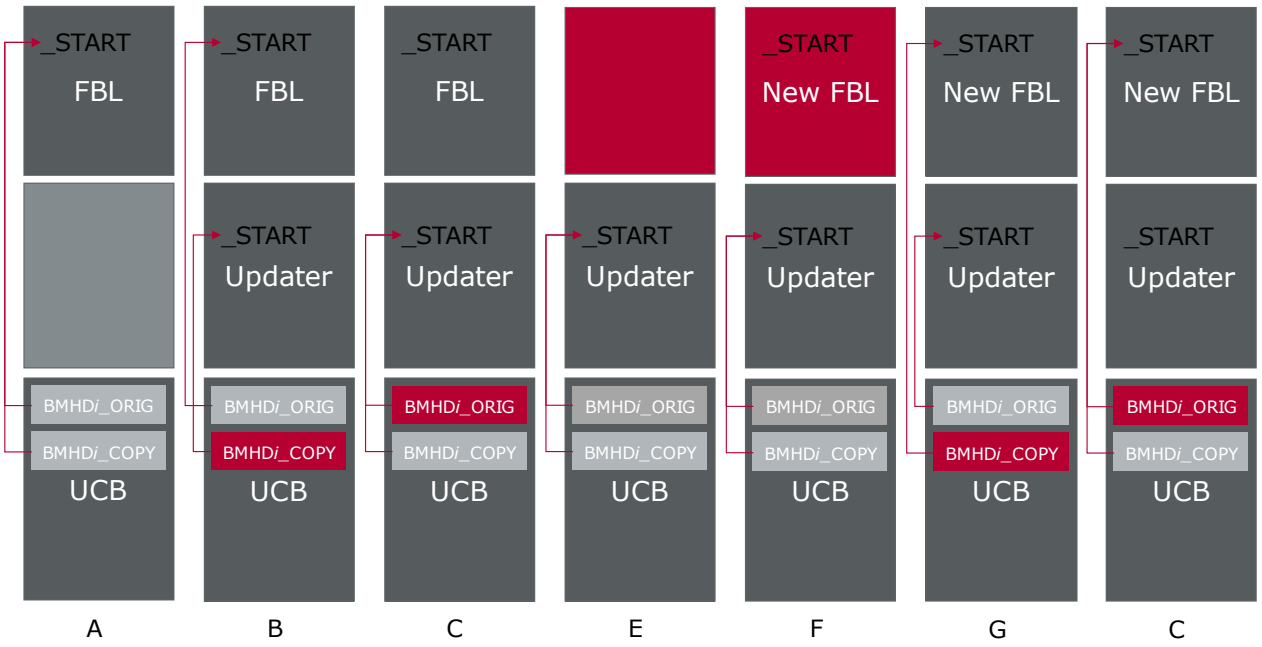


Figure 2-3 Variant A: Replace the first BMHD for Updater

2.3.2 Variant B: Use second BMHD for Updater

A second BMHD is initially set to the Updaters starting address, the first one points to the Bootloaders starting address. Prior to erasing and replacing the Bootloader, the first BMHD is invalidated (e.g. erased) and afterwards validated again.

Drawbacks:

- If the second BMHD is not changed by the Updater, the mapping for the Updater is restricted because its starting address is fixed.
- Erase and writing in the UCB area is necessary required which can cause a bricked device if the configuration is not completely right..

© 2018 Vector Informatik GmbH

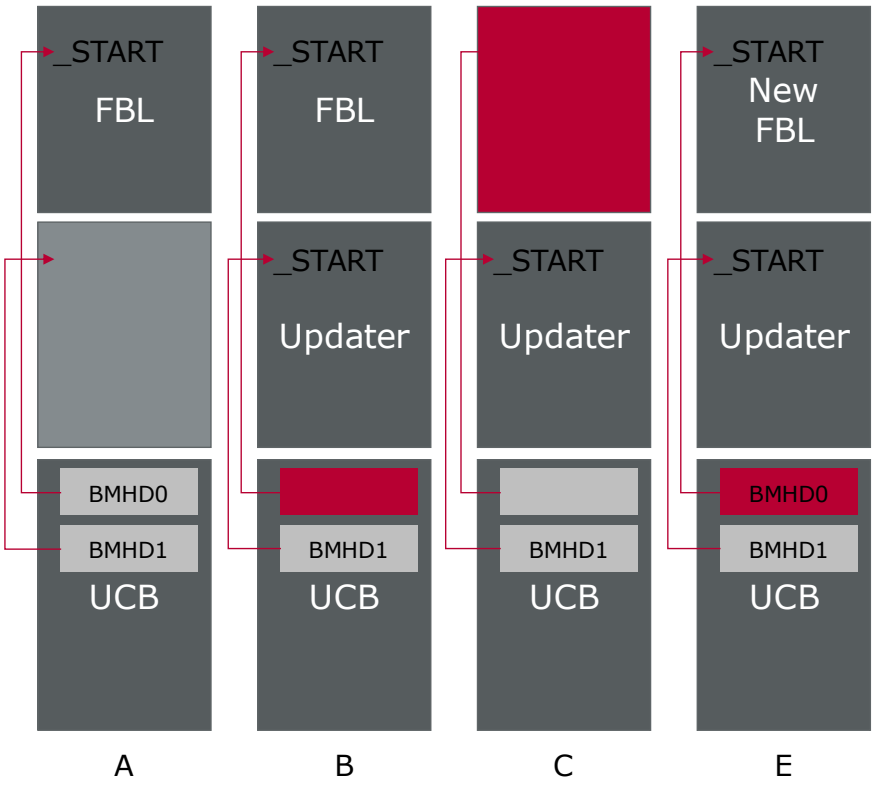


Figure 2-4 Variant B: Use second BMHD for Updater

2.3.3 Variant C: Use ABM for FBL

The first BMHD is initially configured to a certain address in the FBLs flash region. The BOOT_CFG of this BMHD is set to ABM. The second BMHD is initially defined to the Updaters starting address.

The FBL contains a valid ABMHD data structure at the address where the first BMHD points to. If the firmware detects this ABMHD structure, it will directly execute the FBL on startup. If not, it will jump to the Updaters starting address.

During Update Process, the ABMHD is erased first and written after the complete FBL is replaced (“Validity Range”).

The main benefit of this solution is, that a reconfiguration of the UCB memory is not necessary. The main drawback is that the mapping of Updater and FBL is restricted by that.

© 2018 Vector Informatik GmbH

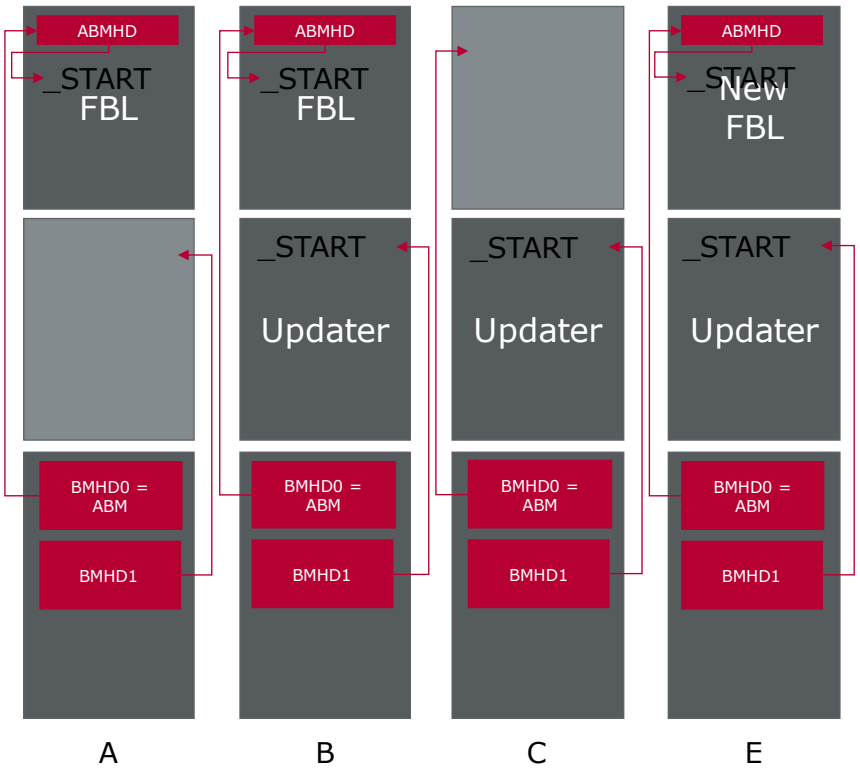


Figure 2-5 Variant C use ABMHD for Bootloader

3 Integration

3.1 Addressing

The Tricore uses an addressing scheme which basically allows to access the same flash region by two addresses. E.g. the access to `0x8000 1234` (cached) can also be done via `0xA000 1234` (uncached).

To create the new FBL's image please make sure to remap all data into the appropriate (uncached addresses) memory region. The Demo Updater Project contains the batch file `FblUpd_Prepare_Hw.bat` which uses Hexview to do such a remapping.

3.2 Initializations



Caution
Ensure that the hardware (e.g.: PLL, GPIOs, CAN, Watchdog …) are initialized at the start of the updater. This is necessary if the updater is (re-) started via its BMHD after a interrupted download process.

The code in the Demonstration Project is only used for demonstration and may not be suitable for your hardware and has to be replaced.



Caution
Some configuration details of the hardware cannot be initialized twice. If initializations are done at first by the FBL and again by the Updater, unexpected behavior like continued resets (traps) may be observed if the Updater is called from the FBL.

3.3 Hooks

To activate the reset-safe behavior, the hooks

- `FBL_UPD_HOOK_PREPARE_ERASE` → `ApplFblUpdHwSetBmhdToUpdater()`
- `FBL_UPD_HOOK_FINALIZE_VERIFY` → `ApplFblUpdHwSetBmhdToFbl()`

Need to be attached. The first one is called prior to erasing the Bootloader, the second one after successful update process.

To enable the reset-safety, define:

```
#define FBL_UPD_HOOK_PREPARE_ERASE   ApplFblUpdHwSetBmhdToUpdater
#define FBL_UPD_HOOK_FINALIZE_VERIFY ApplFblUpdHwSetBmhdToFbl
```

in `upd_cfg.h` to attach to the hooks.

Caution
It is strongly recommended to set up the Updater without reset-safety for testing. If everything works, enable reset-safety. This will avoid side effects caused by debugging which may result in a corrupted UCB content.

3.4 BMHD content

The easiest way to get a valid BMHD content is to extract it with the debugger from your device. This can be done with e.g.

- Lauterbach Debugger:
  `Data.save.intelhex c:\UCB_BMHD0.hex 0xAF400000--0xAF4001FF`

The relevant content of the BMHD is shown in Figure 3-1. Beside of these, the BMHD also contain some password values which may restrict the access to the UCB block.

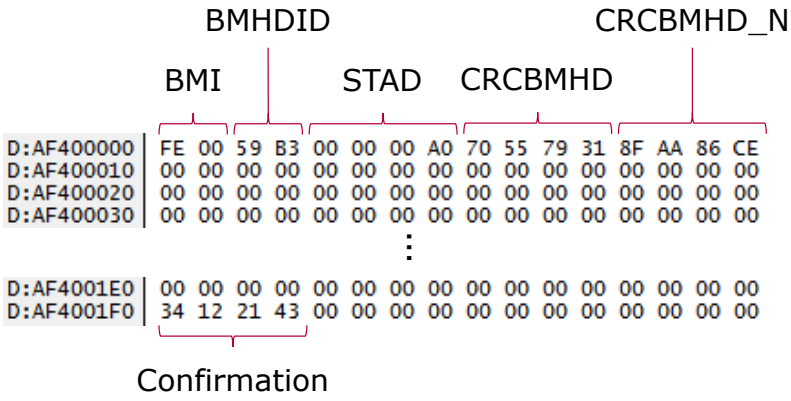


Figure 3-1 Typical BMHD content.

The field CRCBMHD contains the CRC32 checksum over BMI, BMHDID and STAD. CRCBMHD_N contains the bitwise inverse of this value.

Pseudocode:

```
CRCBMHD = CRC32(0xAF400000, 8)
CRCBMHD_N = ~CRCBMHD
```

3.4.1 BMHD Generator Script

Please have a look at the batch file Generate_BMHD.bat It asks for the Startup Address (STAD) and if this shall be the entry for

1. Updater
2. FBL

It generates the files `BMHD_ToFbl.c/.h` resp. `BMHD_ToUpdater.c/.h`, depending on the selection. These files contain C-Array definitions with a valid BMHD content.

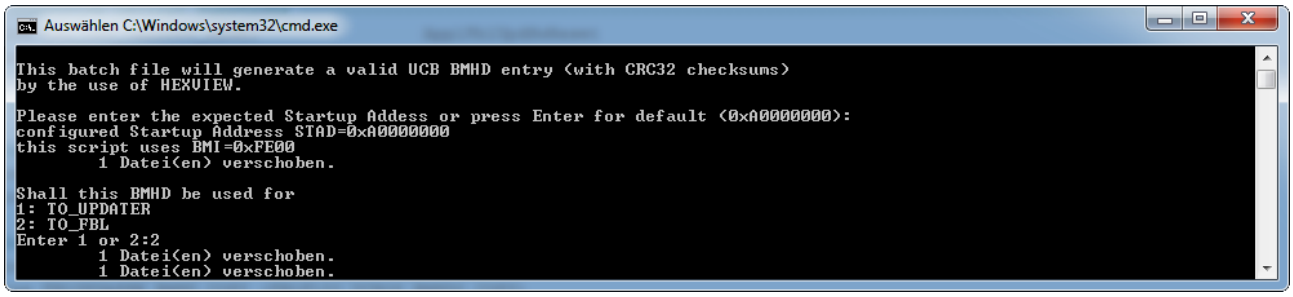


Figure 3-2 Screenshot from batch file

The main purpose of the batch file is to generate the right (CRC32) checksum values, which is a bit complicated because of the required big endian ↔ little endian switch.

3.5 Flash Driver settings

The Flash Driver usually does not allow write access to the UCB area. To enable it, use the defines:

```
#define FLASH_ENABLE_TRICORE_UCB_ERASE
#define FLASH_ENABLE_TRICORE_UCB_READ
#define FLASH_ENABLE_TRICORE_UCB_WRITE
#define FLASH_ENABLE_TRICORE_UCB_DISABLE_PROTECTION
```

3.6 Configuration

The configuration can be done by editing upd_hw_cfg.h by uncommenting the prepared defines.

3.7 Demonstration Code

The demonstration code contains a solution for Variant A (see 2.3.1). It is expected as a precondition that at least one BMHD (_ORIG or _COPY) is valid. This is a reasonable expectation because otherwise the device would not start up.

The code detects which BMHD is the active one and if it is in CONFIRMED or UNLOCKED state. Depending on the situation it decides

- If BMHD is CONFIRMED, a Disable Protection sequence is required to allow write access to the BMHD.

  For Disable Protection a password is needed: The found active BMHD is compared to the configured BMHD_TO_FBL and to the BMHD_TO_UPDATER array. Depending on which matches, the password values from the array are used (see Figure 3-3).

- If the BMHD_ORIG is not CONFIRMED or UNLOCKED, the order of writing is inverted: The BMHD_ORIG is written before the BMHD_COPY is updated

In Table 3-1 all possible situations and the reaction of the Updater demonstration setup are listed.

BMHD_ORIG CONFIRMATION	BMHD_COPY CONFIRMATION	Overall Confirmation	Should never happen	Active BMHD	Reaction
ERRORED	ERRORED	ERRORED	■	-	Fail
ERRORED	UNLOCKED	UNLOCKED		COPY	Erase & Write ORIG, Erase & Write COPY
ERRORED	CONFIRMED	CONFIRMED		COPY	Disable Protection, Erase & Write ORIG, Erase & Write COPY
UNLOCKED	ERRORED	UNLOCKED	■	ORIG	Erase& Write COPY, Erase & Write ORIG
UNLOCKED	UNLOCKED	UNLOCKED	■	ORIG	Erase& Write COPY, Erase & Write ORIG
UNLOCKED	CONFIRMED	UNLOCKED		ORIG	Erase& Write COPY, Erase & Write ORIG
CONFIRMED	ERRORED	CONFIRMED	■	ORIG	Disable Protection, Erase & Write COPY, Erase & Write ORIG
CONFIRMED	UNLOCKED	CONFIRMED	■	ORIG	Disable Protection, Erase & Write COPY, Erase & Write ORIG
CONFIRMED	CONFIRMED	CONFIRMED		ORIG	Disable Protection, Erase & Write COPY, Erase & Write ORIG

Table 3-1 Possible (expected) situations on start of the Updater

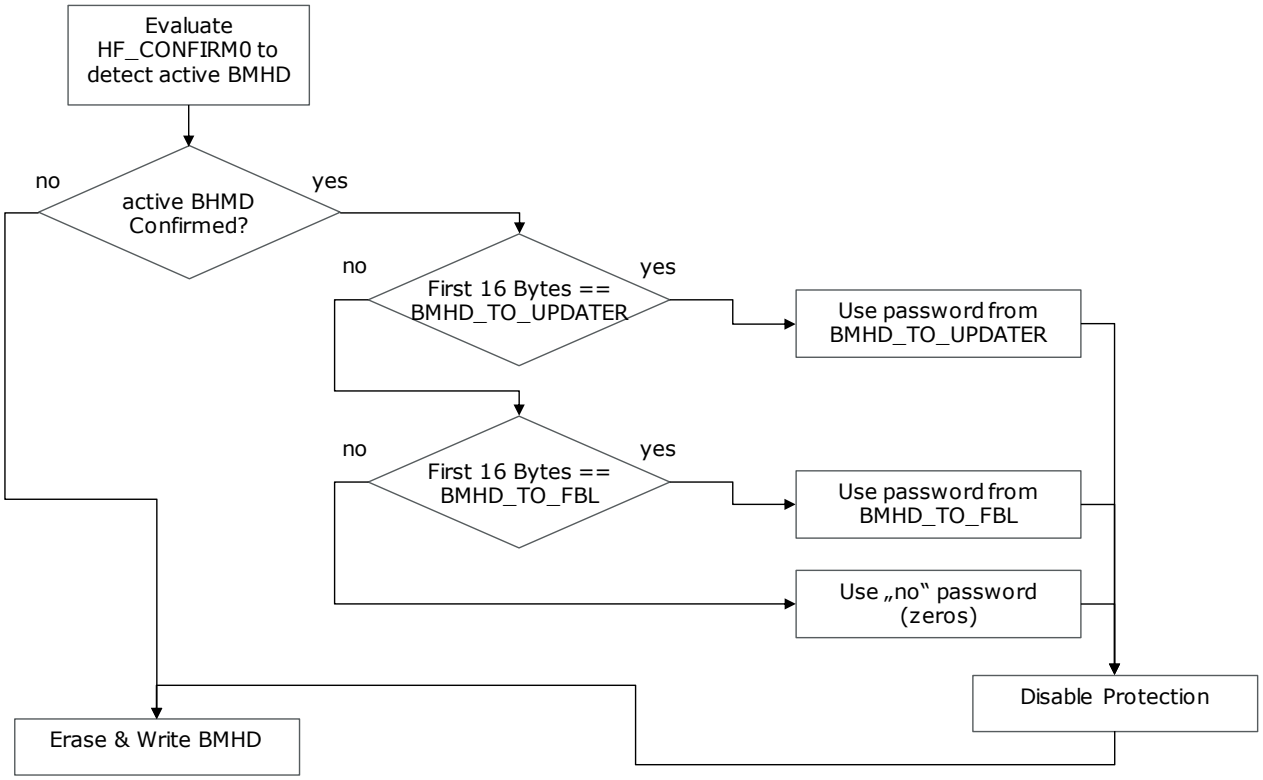


Figure 3-3 Disable Protection flow

4 Abbreviations

Abbreviation	Description
ABM	Alternative Boot Mode
ABMHD	Alternative Boot Mode Header
BMHD	Boot Mode Header
FBL	Flash Bootloader
UCB	User Configuration Block

5 Contact

Visit our website for more information on

- News
- Products
- Demo software
- Support
- Training data
- Addresses

www.vector.com

© 2018 Vector Informatik GmbH